Model-based testing

October 20th, 2025
Ana Paiva, Auri Vincenzi, José Campos

In this recitation class, we are going to explore ‘Model-based Software Testing’, *a black-box testing technique*.

1. State Machine

Suppose there is an air conditioning system named `ColdHot,` which has the following requirements:

- When the user turns it on, the machine is in an idle state.
- If the user turns it off, the machine is off. If the user turns it on again, the machine is back to idle.
- If the room is too hot, then the cooling process starts. It goes back to idle when the defined temperature is reached.
- If the room is too cold, then the heating process starts. It goes back to idle when the defined temperature is reached.

Draw a minimal state machine to represent these requirements. (Tip: You may use draw.io to draw the state machine.) How many states, transitions, and events are there? Write your answer in a .txt file.

2. Transition Tree

Derive the transition tree of the `ColdHot`’s state machine. How many different paths are there? Write your answer in a .txt file.

3. Transition Table

Derive the transition table of the `ColdHot`’s state machine using the transition tree. How many “normal” paths and sneaky paths can be tested based on this transition table? Write your answer in a txt file.

4. Challenge: model-based testing in the `jpacman` project

1. List and describe all (or the most relevant) states in the `jpacman` project (source code available here).
2. Draw the state machine that represents those states.
3. Derive the transition tree of the state machine.
4. Derive the transition table of the state machine using the transition tree.
5. Develop the test cases, using the JUnit framework, that exercise all identified paths.

⚠️Note: in Maven-based projects, tests must be developed under `src/test/java`. For more information on how to write JUnit test cases or using Mockito visit the 0. JUnit document or 0. Mockito documento, respectively.

Miscellaneous

- JUnit framework
- JUnit 5 User Guide